---

title: “Protocol Audit Report: PasswordStore”
author: “KAJO”
date: “March 7, 2023”
header-includes:
(- \usepackage{titling} - \usepackage{graphicx})

(\begin{titlepage} \centering \begin{figure}[h] \centering \includegraphics[width=0.5\textwidth]{logo.pdf} \end{figure} \vspace*{2cm} {\Huge\bfseries Protocol Audit Report\par} \vspace{1cm} {\Large Version 1.0\par} \vspace{2cm} {\Large\itshape Cyfrin.io\par} \vfill {\large \today\par} \end{titlepage} \maketitle)

Prepared by: KAJO
Lead Auditors:

- xxxxxxx

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
    - Scope
    - Roles
- Executive Summary
    - Issues Found
- Findings
    - High Severity Issues
    - Medium Severity Issues
    - Low Severity Issues
    - Informational Issues
    - Gas Optimizations

Protocol Summary

PasswordStore is a smart contract protocol dedicated to the secure storage and retrieval of a user's private passwords.

The protocol is strictly designed for single-user operation and is not intended for multi-user environments. Based on the design requirements, **only the contract owner** should possess the authorization to set, update, and access the stored password.

Disclaimer

The **KAJO** security team has made every effort to identify as many vulnerabilities as possible within the designated time frame. However, this audit does not guarantee absolute security, and the team holds no liability for any remaining issues or future exploits.

A security audit by our team serves as a technical evaluation and is **not an endorsement** of the underlying business, model, or product. The review was strictly time-boxed and focused solely on the security aspects of the Solidity implementation within the smart contracts.

Risk Classification

We utilize the standard **CodeHawks** severity matrix to evaluate findings based on their potential **Impact** and **Likelihood** of exploitation.

Likelihood	High Impact	Medium Impact	Low Impact
High	High (H)	High/Medium (H/M)	Medium (M)
Medium	High/Medium (H/M)	Medium (M)	Medium/Low (M/L)
Low	Medium (M)	Medium/Low (M/L)	Low (L)

For further details regarding finding evaluation, please refer to the CodeHawks Severity Documentation.

Audit Details

- **Commit Hash:**
  `0x793be48f89f6628ecd4313375ef186d82478156545a840e29881e309d20887e0`

Scope

The following contract was reviewed during the audit:

- `src/PasswordStore.sol`

Roles

- **Owner:** The sole authorized user who can set and retrieve the password.

Executive Summary

Issues Found

Severity	Number of Issues	Status
High	2	Open
Medium	0	-
Low	0	-
Informational	0	-
Gas	0	-

Findings

High Severity Issues

[H-01] Missing Access Control in `PasswordStore::setPassword` Allows Unauthorized Users to Modify the Password

Classification

- **Severity:** High
- **Impacted Component:** `PasswordStore::setPassword`

Description

The `PasswordStore::setPassword` function completely lacks access control restrictions. Consequently, any external actor (EOA) or smart contract can call this function and overwrite the stored password, even if they are not the designated owner of the contract.

Impact

Critical. This flaw leads to unauthorized state manipulation, resulting in a complete loss of data integrity and allowing attackers to hijack control over the system password.

Proof of Concept (PoC)

The following Foundry test case demonstrates that an arbitrary, non-owner address (`random_address`) can successfully call `setPassword` and alter the contract's state:

```
function testAnyoneCanSetPassword(address random_address) public {
    // Ensure the random address is not the owner
    vm.assume(random_address != owner);

    // Act as the unauthorized user
    vm.prank(random_address);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    // Verify the change as the contract owner
    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation

Implement an explicit access control check within the `setPassword` function using a custom error to ensure only the owner can modify the state:

```
// Define the custom error outside or inside the contract
error PasswordStore__NotOwner();

function setPassword(string memory newPassword) public {
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    s_password = newPassword;
}
```

[H-02] On-Chain Data Visibility Exposes Private Storage Variables (`PasswordStore::s_password`)

Classification

- **Severity:** High
- **Impacted Component:** `PasswordStore::s_password` (Storage Slot)

Description

All data stored on-chain is publicly visible and can be read directly from the blockchain storage slots. Marking the `s_password` variable as `private` only prevents other smart contracts from reading it at compilation time. It does not prevent external users from reading the raw storage slot directly from an RPC node.

Impact

High. The confidentiality of the stored password is completely compromised, as any user can extract the plaintext data directly from the public ledger.

Proof of Concept (PoC)

An attacker can spin up a local network environment and read the storage slot directly using standard development command-line tools:

1. Spin up a local development chain:

```
anvil
```

2. Deploy the contract to the local network:

```
forge create PasswordStore --rpc-url http://127.0.0.1:8545 --private-key <PRIVATE_KEY>
```

3. Use a storage inspection tool (like `cast`) to read storage slot `0` where `s_password` resides:

```
cast storage <CONTRACT_ADDRESS> 0 --rpc-url http://127.0.0.1:8545
```

Recommended Mitigation

Do not store sensitive, unencrypted data like plaintext passwords directly on-chain. If blockchain storage is mandatory, the password must be encrypted off-chain, hashed, or managed using cryptographic commitment schemes (such as Zero-Knowledge Proofs) before submission.

Medium Severity Issues

No medium-severity findings were identified.

Low Severity Issues

No low-severity findings were identified.

Informational Issues

No informational findings were identified.

Gas Optimizations

No gas optimization findings were identified.